

MiST: rapid, accurate and flexible (core-genome) multi-locus sequence typing (MLST) allele calling from draft genomes

Received: 8 September 2025

Accepted: 10 November 2025

Published online: 14 November 2025

Cite this article as: Bogaerts B., Roosens N.H.C. & Vanneste K. MiST: rapid, accurate and flexible (core-genome) multi-locus sequence typing (MLST) allele calling from draft genomes. *BMC Genomics* (2025). <https://doi.org/10.1186/s12864-025-12324-z>

Bert Bogaerts, Nancy H. C. Roosens & Kevin Vanneste

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

APPLICATION NOTE

**MiST: rapid, accurate and flexible (core-genome)
multi-locus sequence typing (MLST) allele calling
from draft genomes**

Bert Bogaerts¹, Nancy H. C. Roosens¹, Kevin Vanneste¹

¹ *Transversal activities in Applied Genomics, Sciensano, Brussels, Belgium*

Corresponding author: Bert Bogaerts (bert.bogaerts@sciensano.be)

Word count (main body of article): 3,998 words

Number of Figures: 3

Number of Tables: 2

Keywords: core genome multi-locus sequence typing (cgMLST), whole genome sequencing, outbreak investigation, strain typing

Abstract

Background

Multi-locus sequence typing (MLST) is a typing method to differentiate bacteria based on the sequence of several housekeeping genes. Identifiers are assigned to unique allele sequences for each locus in the scheme, and the combination of these identifiers defines a genetic profile. Its reproducibility and portability across laboratories have made it a staple typing method, widely used for epidemiological and evolutionary analyses. Although MLST was traditionally based on Sanger sequencing, the method is still widely used in the era of whole-genome sequencing (WGS). Moreover, WGS has made core-genome MLST (cgMLST) possible, which scale up MLST to hundreds or thousands of loci across the genome. Conventional and cgMLST schemes are publicly available on various platforms such as PubMLST.org, BIGSdb Institut Pasteur, cgMLST.org, and Enterobase. However, the available software for (offline) cgMLST allele calling is often not flexible to accommodate schemes from diverse sources and/or lacks the computational scalability required to efficiently process larger schemes.

Results

In this study, we present Minimap2-inferred Sequence Typing (MiST), a rapid and flexible cgMLST allele caller that is low-resource intensive and can easily accommodate schemes from different sources. We benchmarked the tool against other available MLST and cgMLST calling software using WGS data from five different species and schemes collected from various sources. We demonstrate that MiST can accurately identify alleles while requiring substantially fewer computational resources than existing allele callers.

Conclusion

48 MiST can help to make cgMLST analysis more accessible for integration into local
49 bioinformatics workflows. MiST is available as an open-source Python package under the
50 GPLv3 license at <https://github.com/BioinformaticsPlatformWIV-ISP/MiST>, and can be installed
51 via Pip or Conda.

ARTICLE IN PRESS

1. Background

Multi-locus sequence typing (MLST) is a well-established typing method for bacteria [1]. In MLST, a small set of housekeeping genes is sequenced with Sanger sequencing to type bacterial isolates based on their allelic profiles, by comparing against a harmonized reference database where each different sequence receives a different identifier (i.e., a number), providing a standardized way to study genetic diversity and epidemiology [2]. A sequence type (ST) is then assigned to each unique combination of these allele identifiers. Allele sequences and MLST profiles are maintained in public repositories such as PubMLST, which hosts databases for over 140 species and genera [1]. Although MLST was initially designed for typing bacterial strains, it is also used for typing fungi [3] and plasmids [4]. Historically, the lower throughput of Sanger sequencing has limited MLST to a few loci. The onset of whole-genome sequencing (WGS) has rendered it possible to scale MLST to hundreds or thousands of loci across the genome, greatly increasing the resolution. In core-genome MLST (cgMLST), the scheme is restricted to core loci shared by the majority of strains within a species or genus (typically found in $\geq 95\%$ of strains). In contrast, whole genome MLST (wgMLST) includes all annotated loci in the genome, including both core and accessory genes. Compared to cgMLST, the number of available wgMLST schemes is limited. The greatly increased resolution of cgMLST and wgMLST compared to classic MLST enables the differentiation of closely related strains in outbreak and relatedness investigations [5]. Consequently, cgMLST and wgMLST schemes have been developed for many species [6], [7], with cgMLST in particular becoming a staple method for infectious disease surveillance and outbreak investigation [8]. Analogous to conventional MLST, most cgMLST schemes also define core-genome sequence types (cgSTs).

Numerous software packages are available for (cg)MLST analysis, which can be divided into two categories: (1) genome-based tools and (2) read-based tools. Genome-based tools start with draft or complete genomes in FASTA format and typically employ alignment methods to compare them with the allele sequences in the database. In this category,

Seemann-MLST (available at <https://github.com/tseemann/mlst>) and CGE-MLST [9] are two commonly used tools, intended for conventional MLST. Tools developed specifically for cgMLST, such as ChewBBACA [10] and pyMLST [11], generally cluster allele sequences to reduce the number of sequences that need to be aligned, thereby reducing running times. These two tools also offer the possibility to create schemes *de novo* based on a set of input genomes. Recently, CoreProfiler also became available, which was developed specifically for rapid, offline cgMLST calling (<https://gitlab.com/ifb-elixirfr/abromics/coreprofiler>). CoreProfiler currently supports a set of cgMLST schemes hosted on BIGSdb, but no MLST schemes. It uses a two-step BLAST-based approach whereby it performs a rapid screening for perfect matches followed by a slower screening for potential novel alleles. Read-based tools start with read data, typically in FASTQ format, and use read mapping or k-mer matching to compare the reads directly with the sequences in the database. In theory, these methods can be faster, as the *de novo* assembly step is not required. StringMLST [12], MentalIST [13], and ARIBA [14] are some of the available read-based tools. These tools start with an initial mapping step, but ARIBA performs an additional local re-assembly using the reads mapped to the target sequence.

Although several tools are available for accurate MLST calling, scaling these tools up to handle cgMLST schemes is often not feasible due to performance and design limitations. Furthermore, most cgMLST-specific software tends to be inflexible due to assumptions or limitations regarding the alleles and loci that can be typed. Finally, while online platforms such as PubMLST and EnteroBase offer extensive functionality including cgMLST typing, they cannot easily be integrated into automated or high-throughput workflows, which limits their usefulness in scalable, reproducible bioinformatics pipelines.

To address these challenges, we developed Minimap2-inferred Sequence Typing (MiST), an open-source Python package for rapid, flexible, and low-resource (cg/wg)MLST allele calling that can accommodate schemes from multiple sources. We benchmarked MiST against several widely used MLST and cgMLST callers using WGS data from seven different bacterial species. By combining high accuracy, resource efficiency, and broad scheme

108 compatibility, MiST makes (cg)MLST analysis more accessible and easier to integrate into
109 scalable bioinformatics workflows.

ARTICLE IN PRESS

2. Implementation

2.1 Software Implementation

2.1.1 Overview

MiST is written in Python 3 and uses Minimap2 [15] to screen for alleles against a subset of representative alleles (Figure 1). During database creation, identity-based clustering is performed on each locus using CD-HIT v4.6.8 to identify a subset of representative alleles. Alleles of different lengths are always assigned to separate clusters, regardless of sequence identity. An additional correction step is included to extract alleles with an alternative start position (i.e., shifted relative to other members of the cluster), by global alignment of all cluster members using nucmer from the MUMmer v4.0.0 package [16]. The SHA-1 hash for each allele is then computed and stored in a JSON file, alongside locus metadata. Allele calling begins with Minimap2 v2.28 [15] alignment against a database containing all the representative alleles, combined across all loci. The '--all-chain' option is enabled to ensure that all alignments, including suboptimal ones, are retained. The location of the locus in the input draft genome is then determined based on the alignment output. The corresponding sequence is then extracted, and its SHA-1 hash is then compared to a database containing pre-computed hashes of known alleles. If an exact match is found, the corresponding allele is reported, otherwise a search for the best-matching allele is performed.

For imperfect hits, alleles are reported as potential novel alleles if they have $\geq 99\%$ sequence identity (this parameter can be adapted) and the length matches exactly to an existing allele in the database. In this case, the best-matching allele(s) from the database are reported and the novel allele sequence is exported. If no such match is found, the locus is reported as missing. If there are multiple hits, there are several options for handling them. The 'longest' strategy is applied by default, whereby the longest perfect matching allele(s) are reported. If there are multiple longest alleles, they are reported separated by '_' (i.e.,

1__4). Alternatively the ‘all’ strategy can be used where all perfect matches are reported, regardless of length. Finally, the ‘first’ strategy is available, which reports the allele with the lowest allele identifier (i.e., the first allele introduced into the database) in the event of multiple hits.

If the detected allele combination matches a (cg)ST in the database, the corresponding profile is reported along with the corresponding metadata. Otherwise, the best matching (cg)ST is reported along with the percentage of matching loci.

2.1.2 Databases

The tool supports arbitrary scheme definitions which can be provided as a list of FASTA files containing the allele sequences for each locus, along with an optional profiles file in tab-separated values (TSV) format. The allele calling method does not rely on translation or identifying coding sequences (CDSs), so allele sequences do not have to start and end with valid start and/or stop codons. Helper scripts are provided to list and download schemes from BIGSdb (e.g., PubMLST, Institut Pasteur Paris), Enterobase, or cgMLST.org. Note that, from this year onwards, registration is required to access the latest allele sequences and profiles in certain BIGSdb instances (including PubMLST.org and BIGSdb Institut Pasteur Paris). Once the user has registered and provided valid credentials, the BIGSdb helper script can retrieve the most recent alleles and profiles.

2.1.3 Phylogenetic tree estimation

We have included a companion script, ‘mist dists’, that can be used to create allele matrices and distance matrices based on the MiST output. The script combines the (cg)MLST output files and generates an allele matrix that can be optionally filtered to remove low quality datasets and/or loci. The ‘--min-perc-loci’ parameter specifies the minimum percentage of loci that must be present in a sample for it to be included in the allele matrix. The ‘--min-perc-

samples` parameter defines the minimum percentage of samples in which a locus must be detected for it to be retained in the allele matrix. A pairwise allele distance matrix is then generated, which can be used as for input for phylogeny software. If GrapeTree [17] is installed, a minimum spanning tree can be generated in Newick format, otherwise the command to generate the tree using the 'MSTreeV2' method is logged. Potential novel alleles are incorporated into the allele matrix using allele identifiers based on sequence hashes to ensure maximum resolution, even when the novel alleles have not yet been added to the underlying scheme.

2.2 Performance evaluation

2.2.1 Strategy

Tool performance was evaluated by comparing the allele calls with an approach where all loci are aligned separately to the input contigs using BLASTN v2.16.0 [18], an approach that was previously validated for various species [19], [20]. For loci with alleles ≤ 100 bp, the BLASTN 'task' parameter was set to 'blastn-short'. This approach is computationally inefficient, but is expected to yield accurate results as each allele sequence in the database is considered for alignment. In rare cases where multiple alleles were identified, the locus was considered a match if at least one of them matched. Only exact matches were considered. Potential novel alleles were treated as missing alleles because the tools use different strategies to identify and report this type of hit.

The resource usage of the various cgMLST calling tools was evaluated using the benchmark functionality of Snakemake v7.32.4 [21]. The following metrics were used: CPU time ('cpu_time'), execution time ('execution_time'), maximum memory usage ('max_rss'), and total I/O (i.e. the sum of 'io_in' and 'io_out', which are the cumulative numbers of megabytes read and written, respectively). All tools were executed with four threads (if multiple threads were supported by the tool) on the same virtualized Ubuntu 22.04 server

with no other analyses running. The server was equipped with two AMD EPYC 7H12 64-core processors and 1 TB of system memory. Resource usage was not assessed for the MLST analysis due to its minimal computational requirements.

2.2.2 Software

An overview of selected, relevant software tools for (cg)MLST analysis is provided in [Table 1](#). For MLST, the performance of MiST was compared with BIGSdb (online) [6], CoreProfiler v1.1.3 (<https://gitlab.com/ifb-elixirfr/abromics/coreprofiler>), a local installation of CGE-MLST (commit #bb8f019) [9], Seemann-MLST v2.23.0 (<https://github.com/tseemann/mlst>), ARIBA and stringMLST v0.6.3 [12]. All tools were executed with default options. For BIGSdb, only the species with MLST schemes available in public instances as listed in [Table 2](#) were evaluated (i.e., *E. faecalis*, *L. monocytogenes*, *M. tuberculosis*, and *N. meningitidis*). Interestingly, none of the MLST loci, except for the *M. tuberculosis* scheme, were compatible with ChewBBACA [10]. The incompatible alleles did not start with a standard start codon or ended with a standard stop codon. Consequently, ChewBBACA was excluded from the MLST allele calling evaluation. For CoreProfiler, databases were pre-indexed using the ‘db makeblastdb’ command according to the authors’ instructions. Schemes that could not be downloaded by CoreProfiler were manually set up by copying the format of the supported databases. We made a minor change to the CoreProfiler source code to ensure its compatibility with MLST schemes. We shared this adjustment with the tool authors via a GitLab issue.

For cgMLST, the performance of MiST was compared with BIGSdb, ChewBBACA and CoreProfiler. StringMLST could not be evaluated as the database construction failed for the larger schemes due to memory usage, even when substantial memory was provided (i.e., over 100 GB of RAM). ARIBA was also omitted because the execution times for larger cgMLST schemes were prohibitively long. CGE-MLST and Seemann-MLST were also omitted because they do not support cgMLST schemes. For BIGSdb, only the schemes available in public instances were evaluated (i.e., *M. tuberculosis*, *N. meningitidis*, and *Y. enterocolitica*).

The following tools were not considered: (1) Ridom SeqSphere due to its commercial license; (2) SRST2 as it is no longer maintained and depends on the deprecated Python v2.7; (3) pyMLST as it only allows one sequence to be imported per locus, making it impossible to evaluate it on complete external schemas; and (4) MentaLiST as it failed to build due to outdated libraries.

2.2.3 Dataset

The tool was evaluated using datasets from seven species for which both MLST and cgMLST schemes were publicly available ([Table 1](#)). The MLST schemes contained seven or eight loci, while the number of loci in the cgMLST scheme ranged from 501 to 3,002. These schemes were selected to include schemes from various origins, including PubMLST.org, cgMLST.org, EnteroBase, and BIGSdb Institut Pasteur. For each species, ten samples were randomly selected from the AllTheBacteria [22] collection, based on the provided species classifications. The assemblies were download from AllTheBacteria, while the corresponding Illumina reads were downloaded from the European Nucleotide Archive (ENA). Accession numbers are provided in [Supplementary Table S1](#).

3. Results

3.1 Accuracy of MiST for conventional MLST allele calling

[Figure 2](#) shows the accuracy of the allele calling using the evaluated MLST schemes ([Table 2](#)). Overall, performance was high for all tools. The most mismatches were identified in the ARIBA output, which failed to detect the S7 and S8 loci in all *M. tuberculosis* datasets. An additional mismatch was identified in the *xpt* locus in one of the *Enterococcus faecalis* datasets. CGE-MLST did not detect the S12 locus for any of the ten *M. tuberculosis* datasets, while it was correctly picked up by all other methods. The remaining mismatches were: (1) the *gdh* locus

in the SAMN35364752 *E. faecalis* dataset, for which the correct allele was identified by CGE-MLST and Seemann-MLST but flagged as imperfect identity; and (2) the *cat* locus in the *L. monocytogenes* SAMN19571709 dataset which was also incorrectly flagged as imperfect identity by Seemann-MLST. This resulted in overall accuracy of 100% for BIGSdb, CoreProfiler, MiST, and StringMLST, 99.60% for Seemann-MLST, 97.80% for CGE-MLST, and 95.80% for ARIBA. The datasets for which there were mismatches with the CGE-MLST tool were re-analyzed using the online tool, which produced the same results as the offline installation.

3.2 Accuracy of cgMLST allele calling

Figure 2 shows the accuracy of the allele calling using the evaluated cgMLST schemes (Table 2). Overall, BIGSdb and CoreProfiler achieved the highest level of accuracy, matching with the reference method for all loci. However, not all schemes were available in BIGSdb and it could only be evaluated on 43,210 of the 118,080 alleles (36.59%). MiST achieved an accuracy of 99.97% on 118,080 allele calls. There were a total of 31 mismatches, 26 of which were caused by small alleles (ranging from 45 to 63 bp) in the *E. coli* cgMLST scheme. These alleles were not detected up in the initial seed alignment with Minimap2. Note that for the BLAST-based approach, the 'task' parameter also had to be adapted to detect these short alleles (see Section 2.2.1). The remaining five mismatches were also caused by loci where the cluster representative was not detected in the seed alignment step (four for the *Neisseria* scheme, one for the *E. coli* scheme), but these alleles were longer. Although the global identity between the reference allele and the cluster representative was higher than the sequence identity threshold, the alignment failed due to localized variation.

For ChewBBACA, the overall accuracy was 87.06%, though this varied substantially between schemes for different species, with the best performance for the *Listeria* and *Yersinia* schemes and the worst performance for the *Mycobacterium* and *Enterococcus* schemes. The primary cause of mismatched alleles was CDS identification, which often resulted in a different allele being called. This occurred with uncommon start codons (e.g., GTG in the *Yersinia*

scheme) or when multiple start or stop codons were located close to each other in the input assembly. ChewBBACA also struggled to identify short alleles, such as the NEIS1020 *Neisseria* locus. As ChewBBACA was originally designed for creating schemes, it is reasonable to expect that it performs best with schemes that closely follow its design guidelines.

3.3 Execution time and resource usage

Resource usage for different cgMLST allele calling methods is provided in [Figure 3](#). Overall, MiST had the shortest execution time, followed by CoreProfiler and ChewBBACA. On average, allele calling for a complete cgMLST scheme using MiST took 15 seconds per dataset, compared to 36 seconds using CoreProfiler, and 83 seconds using ChewBBACA. For all tools, the execution times were correlated with the size of the input scheme (see [Table 2](#)). Execution times tended to be shortest for the *Y. enterocolitica* and *M. tuberculosis* schemes (501 loci and 744 loci, respectively) and longest for *Salmonella* and *Shigella* spp. (3,002 and 2,513 loci, respectively). For example, querying the *Yersinia enterocolitica* cgMLST scheme using MiST took an average of four seconds, whereas querying the *Salmonella* cgMLST scheme took 28 seconds. The memory usage pattern was similar to the execution times, with MiST having the smallest memory footprint followed by ChewBBACA and CoreProfiler. There was a similar scheme-specific effect, with larger schemes requiring more memory. The maximum memory usage across all datasets was 1,555 MB for MiST, 3,683 MB for ChewBBACA, and 4,081 MB for CoreProfiler. ChewBBACA had the highest I/O usage (310 MB), followed by CoreProfiler (84 MB), and MiST, which had the lowest average read/write I/O usage at 37 MB. In summary, MiST had the lowest execution time, whilst using substantially less memory and less I/O than the other tested tools.

4. Discussion

In this study, we present MiST, an open-source Python package for MLST, cgMLST, and wgMLST allele calling. Using schemes from various species and origins, we demonstrated that MiST is faster and requires less memory than the other tested tools with similar or higher accuracies. MiST performed well across all tested schemes and datasets, consistently achieving highly accurate MLST and cgMLST typing results. Since MiST operates entirely within the nucleotide space, it does not consider the effect on the amino acid sequences. This was a deliberate design choice, since many (cg)MLST schemes contain loci that do not correspond to intact CDSs, i.e. they contain additional flanking sequences or contain irregular start or stop codons. While allele size is theoretically irrelevant for MiST, our performance evaluation showed that performance was lower for very small loci (<80bp). Nevertheless, the tool remains very flexible, enabling typing of loci and schemes which are problematic for other tools. For example: (1) ChewBBACA could not identify loci with unconventional CDSs, such as the CTG start codons in the *Yersinia* cgMLST scheme; (2) pyMLST restricts scheme creation to a single sequence per locus, preventing the import of complete external schemes; and (3) BIGSdb and EnteroBase are online platforms that only allow queries on the schemes hosted on their sites. They are also complex to setup locally or to query from the command line.

Two of the most commonly used cgMLST allele callers are ChewBBACA and pyMLST, which also have functionality to create novel schemes. These tools are highly performant and enable relatively rapid typing. However, both tools are limited in their ability to use external schemes. For pyMLST, schemes are created using only a single allele per locus, making it incompatible with existing schemes, which usually have numerous allele sequences for each locus. Consequently, this tool was omitted from the performance evaluation. ChewBBACA has similar limitations regarding the use of external schemes. ChewBBACA relies on CDS identification and uses BLASTP (i.e., translated nucleotide sequences) for allele calling. However, some existing MLST and cgMLST schemes contain alleles that do not correspond to (conventional) CDSs. For instance, for six out of seven tested MLST schemes, all loci were incompatible with ChewBBACA. This issue was less apparent for the cgMLST schemes for which all loci were compatible, except for the *Neisseria* NEIS1452 locus. Nevertheless, several

alleles were still flagged as ‘invalid’ alleles and were therefore not detectable (e.g., ~5% of alleles in the *E. coli* cgMLST scheme). Furthermore, many of the cgMLST mismatches reported by ChewBBACA were also caused by the CDS calling step. MiST did not have this limitation and was able to handle all tested MLST and cgMLST schemes from various sources without any issues.

Overall, the recently released CoreProfiler tool and the online BIGSdb portal performed very well in our benchmark, matching the reference method for all allele calls and achieving perfect accuracy. BIGSdb also offers extensive functionality, such as novel allele detection and phylogenomic investigation across strains. However, as the portal is hosted online, integrating it into automated workflows is challenging. Furthermore, BIGSdb is restricted to the schemes that are hosted on the platform. MiST does not have these limitations and can be used offline with arbitrary schemes. Compared to MiST, CoreProfiler also achieved slightly higher accuracy, primarily because it had no mismatches in the very short loci of the *E. coli* cgMLST scheme. However, since these loci are rare and limited to a very small number of schemes, this difference does not typically impact standard analyses. MiST could still provide an added value over CoreProfiler. First, CoreProfiler currently supports only a limited set of schemes hosted on BIGSdb instances, and its helper scripts currently do not handle authorization. Consequently, recently added alleles and profiles cannot be retrieved due to changes in PubMLST.org’s data access policy. Second, MiST was less resource-intensive in terms of CPU, RAM and I/O usage. Third, CoreProfiler does not report all matching alleles in cases of multi-hits, i.e. when multiple perfectly matching alleles are present [23]. Although these cases typically arise from sub-optimal scheme curation, they are quite common for the EnteroBase *E. coli* and *Salmonella* cgMLST schemes. For our evaluation we considered a locus a correct match if at least one of the reported alleles matched the reference method (e.g., reporting allele 3 when both alleles 3 and 15 were both present was accepted). However, this behavior can affect downstream phylogenetic analyses, as discrepancies in reported alleles between strains can artificially inflate distances. MiST addresses this issue by reporting multi-hits explicitly using a double-underscore notation (e.g., 3__15).

We also tested several tools that were originally developed for classic MLST, including stringMLST, Seemann-MLST and CGE-MLST. However, these tools either did not support cgMLST schemes (Seeman-MLST and CGE-MLST) or did not scale well to larger schemes (StringMLST). For StringMLST, constructing the database indices caused memory issues, even on high-memory systems. This illustrates the need for optimized, scalable methods for cgMLST analysis.

MiST does not contain functionality to construct novel schemes and is therefore entirely reliant upon externally hosted schemes or other software, such as ChewBBACA. Users are strongly encouraged to deposit novel alleles and profiles into external schemes to ensure they become available to others. MiST applies relatively strict thresholds for reporting potential novel alleles, requiring >99% sequence identity (adjustable by the user) and an exact match in length to an existing allele, which help prevent allele identifiers being assigned to sequencing artefacts or invalid variants while still capturing genuine novel alleles. The user can change the identity thresholds for reporting novel alleles. Stricter values reduce the risk of false positives, but increase the likelihood of genuine novel alleles being missed. Conversely, more relaxed thresholds detect more putative novel alleles, but increase the risk of false positives. When conducting a phylogenomic investigation, it is essential to use the same threshold for all datasets. Potential novel allele sequences can be exported in FASTA format and easily submitted to the database hosting the underlying scheme. The final integration of novel alleles into the underlying scheme is handled by scheme curators, who apply domain-specific knowledge and criteria to accept or reject new allele sequences. MiST displays a reminder to cite the scheme used, which could help to secure funding and continued hosting from the providers of the scheme. Another limitation is the lack of support for raw read input. The *de novo* assembly step can be time-consuming and may affect typing accuracy. However, by starting from FASTA input, the tool can accommodate data from various sources without requiring specialized preprocessing.

5. Conclusion

In conclusion, we have developed an efficient and accurate MLST and cgMLST tool that can enhance the existing (cg)MLST analysis in bioinformatics workflows by providing accurate and resource-efficient (cg)MLST profiling. Furthermore, by providing the ability to download, query and construct phylogenies from (cg)MLST schemes, MiST renders offline cgMLST and wgMLST analysis more accessible.

ARTICLE IN PRESS

Availability and Requirements

Project name: MiST

Project home page: <https://github.com/BioinformaticsPlatformWIV-ISP/MiST>

Operating system(s): UNIX

Programming language: Python 3

Other requirements: Minimap2, CD-HIT, MUMmer4

License: e.g. GNU GPLv3

MiST is available as a Python package that can be installed via 'pip'. It requires Minimap2 [15], CD-HIT [24] and MUMmer4 [16] as external dependencies. Alternatively, MiST and all its dependencies can be installed via Bioconda. The tool can also be used via the Galaxy instance of our institute (registration required) at <https://galaxy.sciensano.be> [25]. Further documentation, including step-by-step examples such as performing an outbreak investigation, is available on the Wiki (<https://github.com/BioinformaticsPlatformWIV-ISP/MiST/wiki>).

List of abbreviations

- Multi-locus sequence typing (MLST)
- Sequence type (ST)
- Core-genome MLST (cgMLST)
- Whole-genome MLST (wgMLST)
- Core-genome sequence type (cgST)
- Coding sequence (CDS)

Declarations**Ethics approval and consent to participate**

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

All data used in this study is available in public repositories ([Table S1](#)).

Authors' contributions

Conceptualization: BB, KV

Data curation: BB

Formal analysis: BB

Funding acquisition: KV, NR

Investigation: BB

Methodology: BB

Project administration: KV

Software: BB

Supervision: KV

Writing – Original Draft: BB

Writing – Review & Editing: BB, NR, KV

Competing interests

The authors declare that they have no competing interests.

Acknowledgements

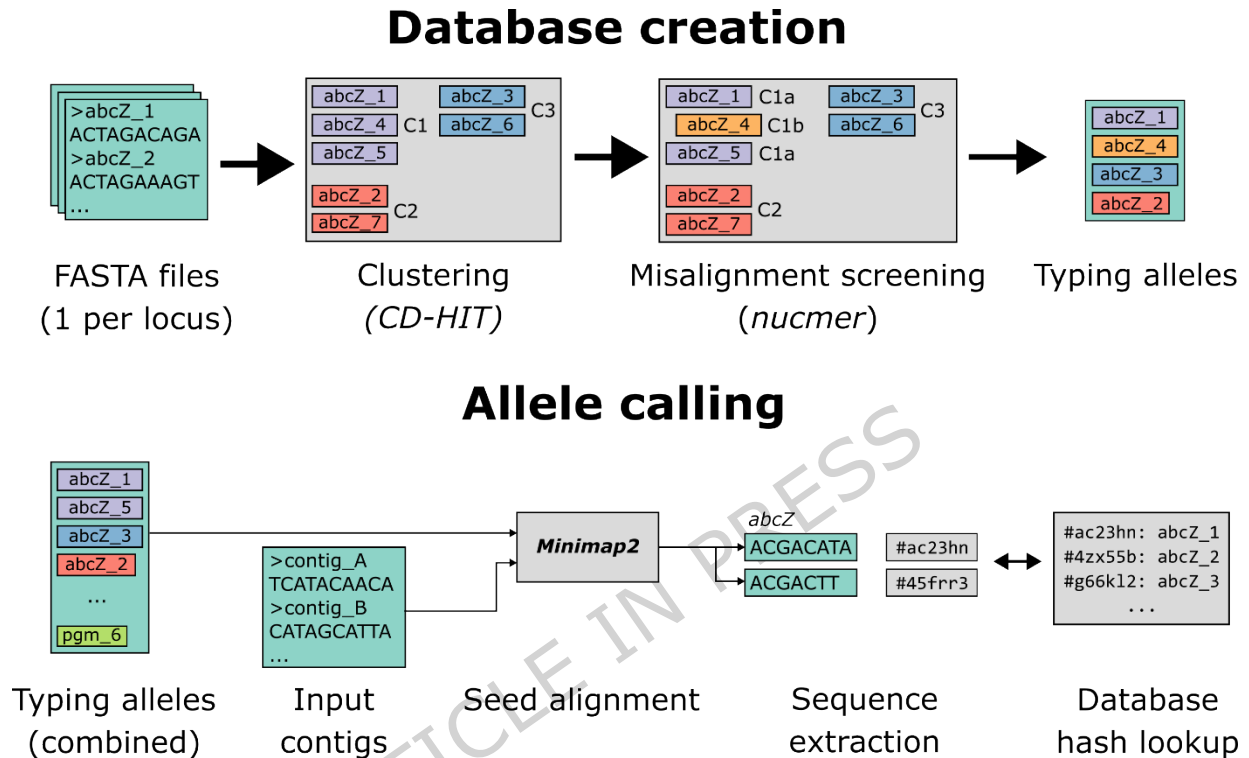
We would like to thank the maintainers of PubMLST.org, BIGSdb Pasteur, cgMLST.org, and EnteroBase for hosting and curating the MLST and cgMLST databases. We would like to thank Torsten Seeman for his feedback on the MiST command line interface and Jane Hawkey and Ryan Wick for their feedback on the hashing of novel allele sequences and sequence types, as well as their general feedback on the tool. We would also like to thank all other people who have provided feedback, either privately or via GitHub issues

Funding

This work was supported by the project “NGS and Bioinformatics Platform” funded by Sciensano (Sciensano RP-PJ, Belgium).

Figures

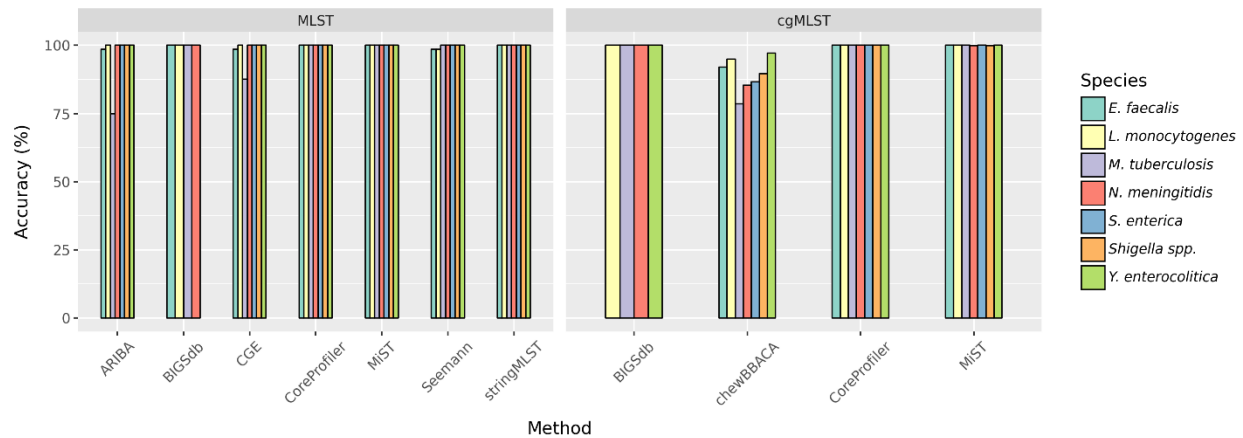
Figure 1: Database construction and allele calling workflow



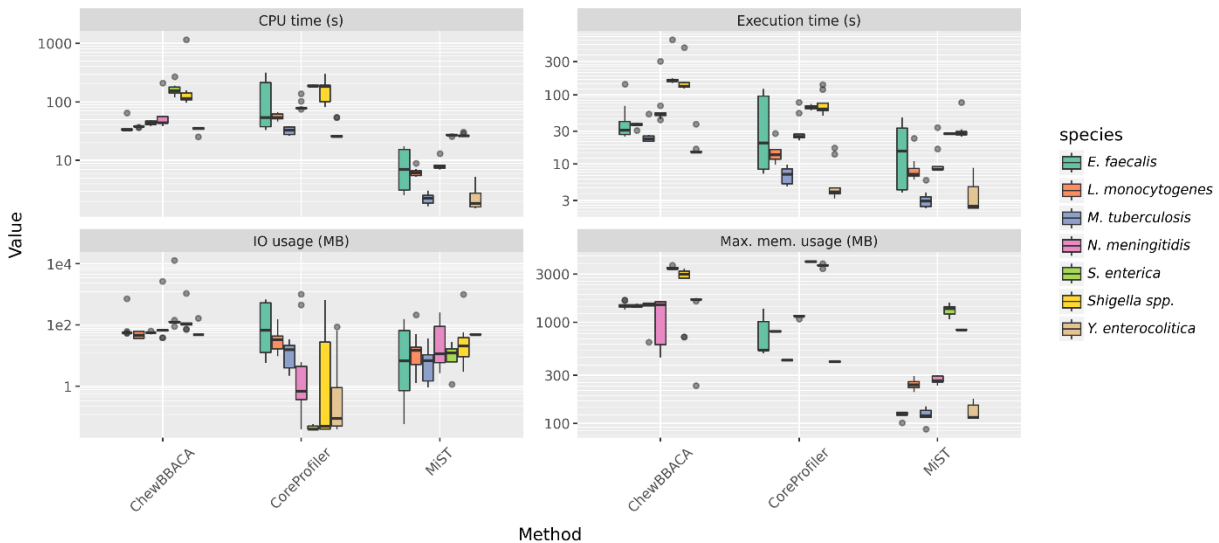
This figure shows the database construction and allele calling strategies. During the database creation, each FASTA file containing allele sequences for a locus is first clustered on sequence identity using CD-HIT. Sequences of different lengths are forced into separate clusters, regardless of identity. The identified clusters are labelled C1, C2, and C3 in this example. Afterwards, alleles with an alternative start position to other members in the cluster are detected using nucmer, and split up into separate clusters (i.e., C1 is split up into C1a and C1b in the example). Finally, all typing alleles (i.e., one representative per cluster) are retained in the final FASTA file. These FASTA files are then combined for all loci and a Minimap2 index is built. The allele calling begins with the alignment of the input contigs to the combined typing alleles using Minimap2. The corresponding sequences are then extracted based on their

482 location in the input contigs. These sequences are then hashed and compared against a
483 database that contains pre-computed hashes for all alleles in the scheme. If one or more exact
484 matches are found, these are reported, otherwise the best-matching allele is identified.

ARTICLE IN PRESS

Figure 2: Allele calling accuracy for MLST and cgMLST

This figure shows the accuracy of allele calling for the various methods for both MLST and cgMLST on the validation dataset. The corresponding schemes are listed in Table 2. The y-axis shows the accuracy, i.e. the percentage of alleles that were called correctly. The x-axis shows the allele calling method. ChewBBACA was not evaluated for the MLST schemes as it was incompatible. StringMLST and ARIBA were not evaluated for the cgMLST schemes as they were unfeasible to run on these large schemes (see Section 2.2.2). BIGSdb could only be used to evaluate schemes that were available on the online portals. The corresponding performance values are listed in Table S2.

Figure 3: Resource usage for different cgMLST allele calling methods

This figure shows the resource usage of the cgMLST allele calling methods on the validation datasets. The top left panel shows the CPU time in seconds, the top right panel shows the execution time in seconds, the lower left panel shows the total I/O usage in MB, and the bottom right panel shows the maximum memory usage in MB. The x-axis shows the allele calling method. The y-axis is on a logarithmic scale and varies between subplots. Dots are colored by species. Resource usage for BIGSdb could not be evaluated as it is an online portal.

Tables

Table 1: Overview of MLST and cgMLST calling software

Tool name	Import schemes	Create schemes	Novel allele detection	Input type	Open source / commercial	Strategy
ARIBA [14]	Yes	No	Yes	Reads	Open source	CD-HIT clustering + read mapping + local assembly
BIGSdb (web-portal) [6]	No	No	Yes	Contigs	Open source	BLAST-based
CGE-MLST [9]	Yes	No	Yes	Contigs / reads	Open source	BLAST-based (FASTA) / KMA-based (FASTQ)
ChewBBACA [10]	Yes ⁽¹⁾	Yes	Yes	Contigs	Open source	CDS calling + BLASTP
CoreProfiler ⁽²⁾	Yes	No	Yes	Contigs	Open source	BLAST-based
Mentalist [13]	Yes	No	Yes	Reads	Open source	Kmer matching
MiST	Yes	No	Yes	Contigs	Open source	CD-HIT clustering + Minimap2
pyMLST [11]	No ⁽³⁾	Yes	Yes	Contigs / reads	Open source	BLAT + MAFFT alignment
Ridom SeqSphere+	No	No	Yes	Contigs / reads	Commercial	n/a
Seemann-MLST ⁽⁴⁾	Yes	No	Yes	Contigs	Open source	BLAST-based
SRST2 [26]	Yes	No	Yes	Reads (Illumina only)	Open source	Bowtie2-based
stringMLST [12]	Yes ⁽⁵⁾	No	No	Reads	Open source	Kmer matching

This table shows the selected (cg)MLST calling software packages. Notes: (1) ChewBBACA does not support all loci and/or alleles; (2) available at <https://gitlab.com/ifb-elixirfr/abromics/coreprofiler>; (3) pyMLST supports importing schemes, but only a single allele can be provided per locus; (4) available at <https://github.com/tseemann/mlst>; (5) stringMLST enables creating schemes, but was infeasible for some larger schemes due to the huge memory requirements for constructing database indices. Note that CGE-MLST and Seemann-MLST do not support cgMLST schemes.

515 Abbreviations: not available (n/a), Center for Genomic Epidemiology (CGE).

ARTICLE IN PRESS

Table 2: Overview of evaluated schemes

Species	Scheme	Origin	Nb. loci
<i>Enterococcus faecalis</i>	MLST	PubMLST.org	7
	cgMLST	cgmlst.org	1,972
<i>Listeria monocytogenes</i>	MLST	BIGSdb Insitut Pasteur Paris	7
	cgMLST	BIGSdb Insitut Pasteur Paris	1,748
<i>Mycobacterium tuberculosis</i>	MLST	PubMLST.org	8
	cgMLST	PubMLST.org	744
<i>Neisseria meningitidis</i>	MLST	PubMLST.org	7
	cgMLST v3	PubMLST.org	1,329
<i>Salmonella enterica</i>	MLST Achtman	EnteroBase	7
	cgMLST	EnteroBase	3,002
<i>Shigella spp.</i>	MLST Achtman (<i>E. coli</i>)	EnteroBase	7
	cgMLST (<i>E. coli</i>)	EnteroBase	2,513
<i>Yersinia enterocolitica</i>	MLST McNally	EnteroBase	7
	cgMLST species	BIGSdb Institut Pasteur Paris	501

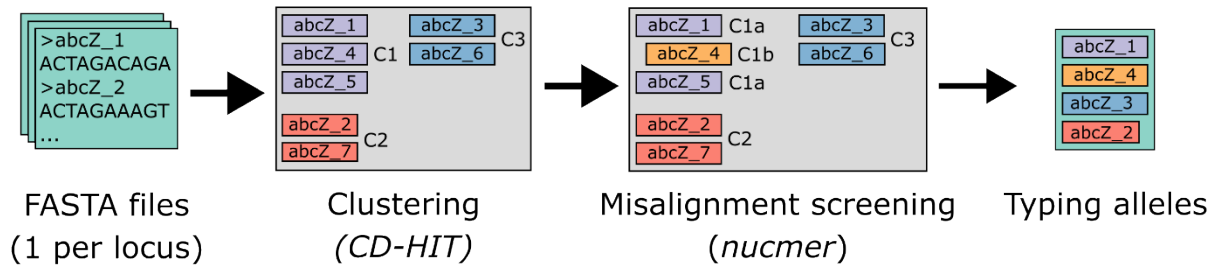
This table provides an overview of the schemes that were used to evaluate the tool performance. Note that the *Escherichia coli* schemes were used for the *Shigella spp.* datasets.

References

- [1] K. A. Jolley, J. E. Bray, and M. C. J. Maiden, "Open-access bacterial population genomics: BIGSdb software, the PubMLST.org website and their applications," *Wellcome Open Res.*, vol. 3, p. 124, Sep. 2018, doi: 10.12688/wellcomeopenres.14826.1.
- [2] M. C. J. Maiden *et al.*, "MLST revisited: the gene-by-gene approach to bacterial genomics," *Nat. Rev. Microbiol.*, vol. 11, no. 10, pp. 728–736, Oct. 2013, doi: 10.1038/nrmicro3093.
- [3] Y.-B. Gong *et al.*, "Multilocus sequence typing of *Candida albicans* isolates from the oral cavities of patients undergoing haemodialysis," *Sci. Rep.*, vol. 8, no. 1, p. 16413, Nov. 2018, doi: 10.1038/s41598-018-34565-7.
- [4] A. Carattoli *et al.*, "In Silico Detection and Typing of Plasmids using PlasmidFinder and Plasmid Multilocus Sequence Typing," *Antimicrob. Agents Chemother.*, vol. 58, no. 7, pp. 3895–3903, Jul. 2014, doi: 10.1128/AAC.02412-14.
- [5] C. Deneke, L. Uelze, H. Brendebach, S. H. Tausch, and B. Malorny, "Decentralized Investigation of Bacterial Outbreaks Based on Hashed cgMLST," *Front. Microbiol.*, vol. 12, p. 649517, May 2021, doi: 10.3389/fmicb.2021.649517.
- [6] K. A. Jolley and M. C. Maiden, "BIGSdb: Scalable analysis of bacterial genome variation at the population level," *BMC Bioinformatics*, vol. 11, no. 1, p. 595, Dec. 2010, doi: 10.1186/1471-2105-11-595.
- [7] Z. Zhou, N.-F. Alikhan, K. Mohamed, Y. Fan, the Agama Study Group, and M. Achtman, "The Enterobase user's guide, with case studies on *Salmonella* transmissions, *Yersinia pestis* phylogeny, and *Escherichia* core genomic diversity," *Genome Res.*, vol. 30, no. 1, pp. 138–152, Jan. 2020, doi: 10.1101/gr.251678.119.
- [8] A. Moura *et al.*, "Real-Time Whole-Genome Sequencing for Surveillance of *Listeria monocytogenes*, France," *Emerg. Infect. Dis.*, vol. 23, no. 9, pp. 1462–1470, Sep. 2017, doi: 10.3201/eid2309.170336.
- [9] M. V. Larsen *et al.*, "Multilocus Sequence Typing of Total-Genome-Sequenced Bacteria," *J. Clin. Microbiol.*, vol. 50, no. 4, pp. 1355–1361, Apr. 2012, doi: 10.1128/JCM.06094-11.
- [10] M. Silva *et al.*, "chewBBACA: A complete suite for gene-by-gene schema creation and strain identification," *Microb. Genomics*, vol. 4, no. 3, Mar. 2018, doi: 10.1099/mgen.0.000166.
- [11] A. Biguenet, A. Bordy, A. Atchon, D. Hocquet, and B. Valot, "Introduction and benchmarking of pyMLST: open-source software for assessing bacterial clonality using core genome MLST," *Microb. Genomics*, vol. 9, no. 11, Nov. 2023, doi: 10.1099/mgen.0.001126.
- [12] A. Gupta, I. K. Jordan, and L. Rishishwar, "stringMLST: a fast k-mer based tool for multilocus sequence typing," *Bioinformatics*, vol. 33, no. 1, pp. 119–121, Jan. 2017, doi: 10.1093/bioinformatics/btw586.
- [13] P. Feijao *et al.*, "MentaLiST - A fast MLST caller for large MLST schemes," *Microb. Genomics*.
- [14] M. Hunt *et al.*, "ARIBA: rapid antimicrobial resistance genotyping directly from sequencing reads," *Microb. Genomics*, vol. 3, no. 10, Oct. 2017, doi: 10.1099/mgen.0.000131.
- [15] H. Li, "New strategies to improve minimap2 alignment accuracy," *Bioinformatics*, vol. 37, no. 23, pp. 4572–4574, Dec. 2021, doi: 10.1093/bioinformatics/btab705.

- [16] G. Marçais, A. L. Delcher, A. M. Phillippy, R. Coston, S. L. Salzberg, and A. Zimin, "MUMmer4: A fast and versatile genome alignment system," *PLOS Comput. Biol.*, vol. 14, no. 1, p. e1005944, Jan. 2018, doi: 10.1371/journal.pcbi.1005944.
- [17] Z. Zhou *et al.*, "GrapeTree: visualization of core genomic relationships among 100,000 bacterial pathogens," *Genome Res.*, vol. 28, no. 9, pp. 1395–1404, Sep. 2018, doi: 10.1101/gr.232397.117.
- [18] C. Camacho *et al.*, "BLAST+: architecture and applications," *BMC Bioinformatics*, vol. 10, no. 1, p. 421, Dec. 2009, doi: 10.1186/1471-2105-10-421.
- [19] B. Bogaerts *et al.*, "Validation of a Bioinformatics Workflow for Routine Analysis of Whole-Genome Sequencing Data and Related Challenges for Pathogen Typing in a European National Reference Center: *Neisseria meningitidis* as a Proof-of-Concept," *Front. Microbiol.*, vol. 10, p. 362, Mar. 2019, doi: 10.3389/fmicb.2019.00362.
- [20] B. Bogaerts *et al.*, "Validation strategy of a bioinformatics whole genome sequencing workflow for Shiga toxin-producing *Escherichia coli* using a reference collection extensively characterized with conventional methods," *Microb. Genomics*, vol. 7, no. 3, Mar. 2021, doi: 10.1099/mgen.0.000531.
- [21] J. Köster and S. Rahmann, "Snakemake—a scalable bioinformatics workflow engine," *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, Oct. 2012, doi: 10.1093/bioinformatics/bts480.
- [22] M. Hunt *et al.*, "AllTheBacteria - all bacterial genomes assembled, available and searchable," Mar. 11, 2024. doi: 10.1101/2024.03.08.584059.
- [23] B. Bogaerts *et al.*, "Oxford Nanopore Technologies R10 sequencing enables accurate cgMLST-based bacterial outbreak investigation of *Neisseria meningitidis* and *Salmonella enterica* when accounting for methylation-related errors," *J. Clin. Microbiol.*, pp. e00410–25, Aug. 2025, doi: 10.1128/jcm.00410-25.
- [24] W. Li and A. Godzik, "Cd-hit: A fast program for clustering and comparing large sets of protein or nucleotide sequences," *Bioinformatics*, vol. 22, no. 13, pp. 1658–1659, 2006, doi: 10.1093/bioinformatics/btl158.
- [25] B. Bogaerts *et al.*, "Galaxy @Sciensano: a comprehensive bioinformatics portal for genomics-based microbial typing, characterization, and outbreak detection," *BMC Genomics*, vol. 26, no. 1, p. 20, Jan. 2025, doi: 10.1186/s12864-024-11182-5.
- [26] M. Inouye *et al.*, "SRST2: Rapid genomic surveillance for public health and hospital microbiology labs," *Genome Med.*, vol. 6, no. 11, p. 90, Dec. 2014, doi: 10.1186/s13073-014-0090-6.

Database creation



Allele calling

